Context Compression

for Large Language Models

Julia Belikova

Context is the bottleneck

Bigger context windows are advertised everywhere — but the cost and the usable fraction tell another story.

$O(n^2)$

attention cost in sequence length n

~1–2M

tokens: largest context windows (2026)

50–65%

of that window is actually used well

Two hard limits behind the hype

- Attention compares every token with every other → quadratic time & memory
- The KV cache grows linearly with context → dominates inference memory/latency
- Even 1M-token models over-attend to the start/end ("lost in the middle")
- **So: reduce the tokens, keep the signal — that is context compression**

Frontier mid-2026 (leaderboards shift monthly): Claude Opus 4.8 · GPT-5.5 · Gemini 3.1 Pro · Grok 4.3 — open: DeepSeek V4, Qwen 3.5, Llama 4, GLM-5, Kimi K2

PART 1

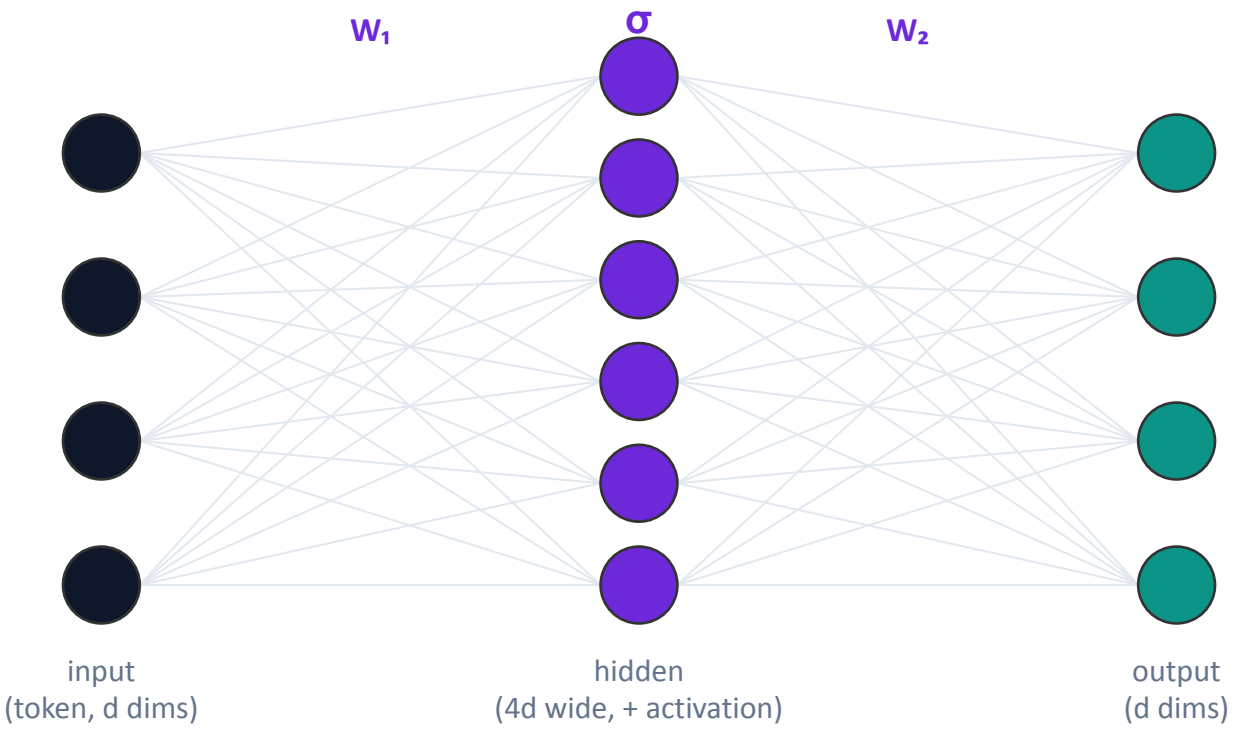
Just-enough LLM foundations

Only the pieces the rest of the talk needs.

- Anatomy of an LLM
- Attention & the $O(n^2)$ cost
- The KV cache
- Training & LoRA

1 The feed-forward network — from the ground up

First the general idea: a small fully-connected net — weighted sums, a nonlinearity, repeat.

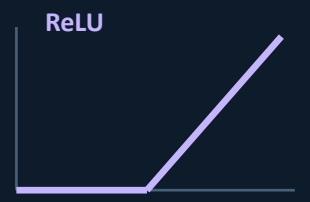


Inside one neuron

$\Sigma\, w_i x_i + b$
weighted sum + bias → σ → a

$$a = \sigma\left(\sum_i w_i x_i + b\right)$$

Why an activation?

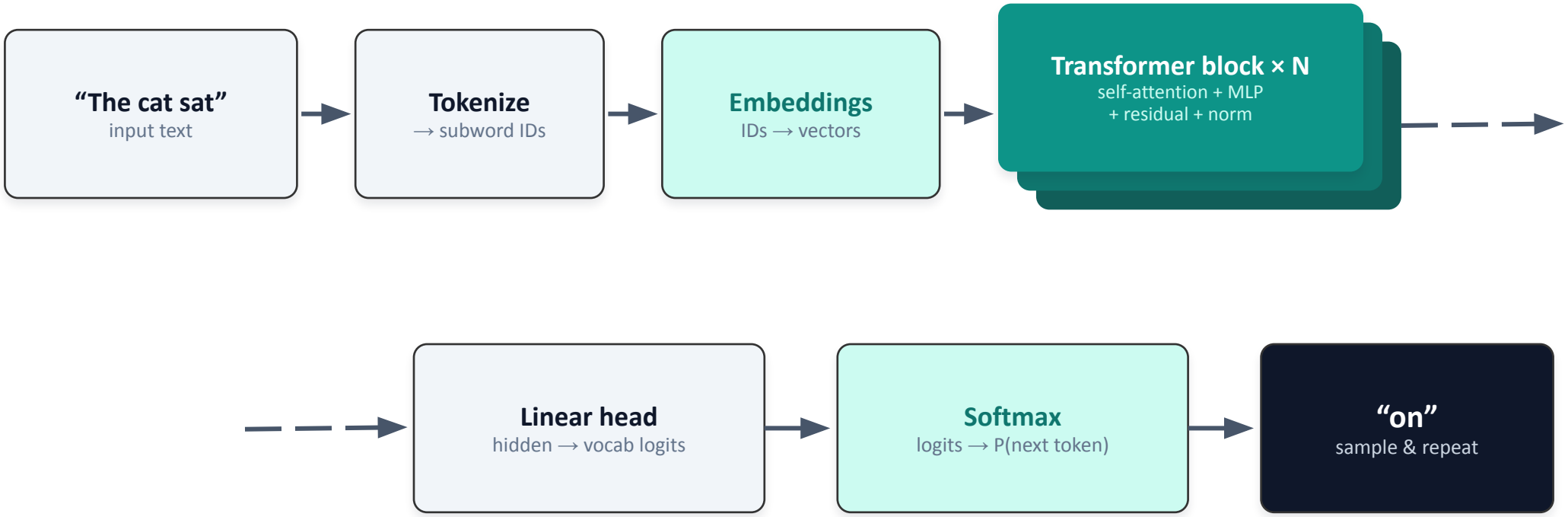


Without a nonlinearity, stacking layers collapses to one linear map. ReLU · GELU · SiLU / SwiGLU add the curvature that makes depth useful.

stack these units ⟶ $FFN(x) = W_2 \cdot \sigma(W_1 x + b_1) + b_2$ (next slide)

1 Anatomy of an LLM

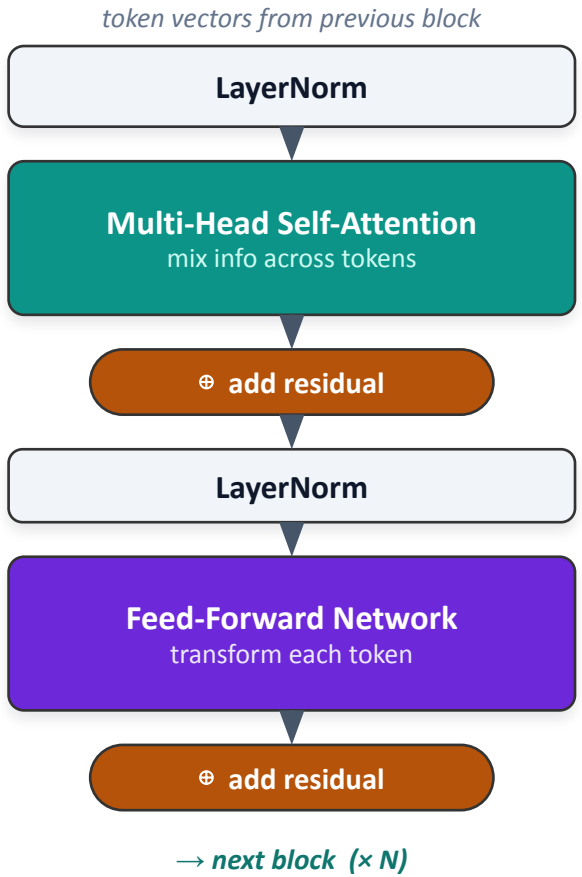
Text becomes vectors, a stack of transformer blocks mixes them, a softmax predicts the next token.



Key idea for later: everything the model "knows" about the current input lives in these token vectors — compression works on exactly this.

1 Inside one transformer block

Two sub-layers, repeated N times: attention mixes tokens, then a feed-forward net transforms each one.



Attention = communication

Every token gathers relevant content from all other tokens (via query/key/value). This is the only place tokens interact.

Feed-forward = computation

The same small MLP runs on each token independently — it holds most of the model's parameters and stored patterns.

Residual + LayerNorm = stability

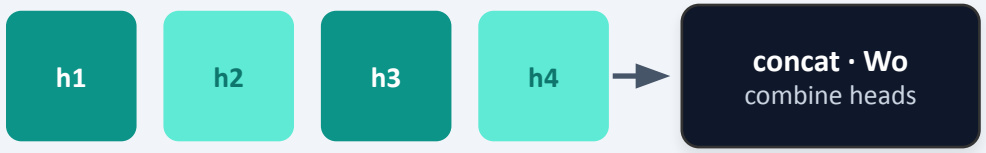
Skip connections carry the input forward and normalization tames the scale, so stacks dozens of layers deep still train.

Rule of thumb: attention moves information sideways; the FFN transforms it in place. Repeat N times.

1 Attention: queries, keys & values

Each token forms a query, matches it against every key, and pulls back a weighted mix of values.

Multi-head attention



Heads run in parallel — each learns a different relation (syntax, coreference, …).

The intuition

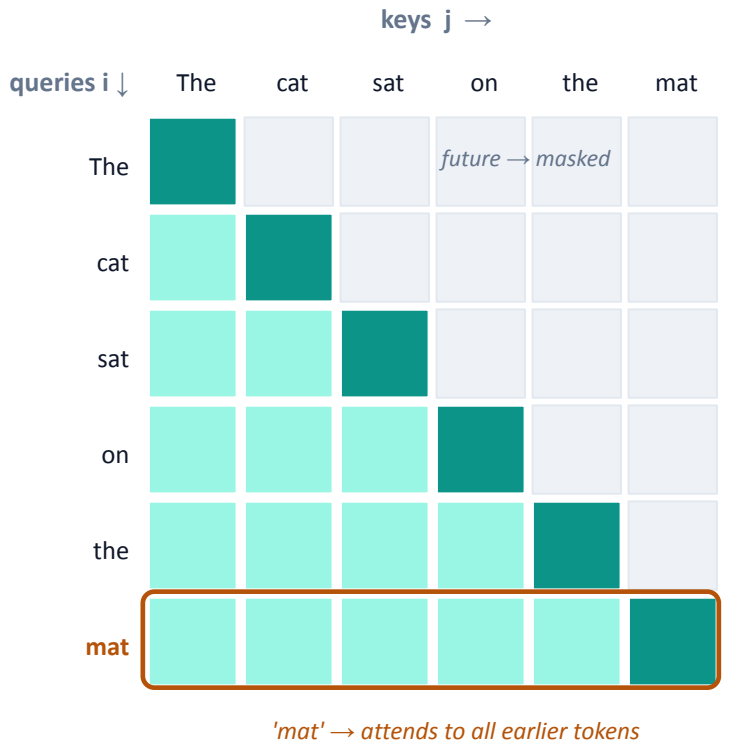
- Query = what this token is looking for
- Key = what each token offers · Value = the content it carries
- softmax turns match scores into weights that sum to 1

1 Attention's cost — why length hurts

That all-pairs comparison has a price: the score matrix grows with the square of the sequence length.

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\mathbf{Q\,K^T / \sqrt{d}} \right) V$$

- $Q\,K^T$ is an $n \times n$ matrix — one score for every ordered pair of tokens
- Time and memory scale as $O(n^2)$: double the context ⟶ 4× the attention cost
- **This is the wall long-context models keep hitting — and the reason to compress**



Causal mask: token i attends only to itself + earlier tokens (lower triangle)
n tokens ⟶ n^2 scores ⟶ $O(n^2)$

1 The feed-forward network — inside the transformer

The same net, now applied to every token independently — where most of the model's parameters live.

hidden is ~4× wider · each unit = σ(Σ w·x + b)

$$FFN(x) = W_2 \cdot \sigma(W_1 x + b_1) + b_2$$

- Position-wise: the same weights run on each token independently — no mixing between tokens happens here
- Usually ~⅔ of all parameters; behaves like a learned key–value memory of patterns
- **Modern LLMs use a gated SwiGLU variant; Mixture-of-Experts swaps in many expert FFNs and routes each token**

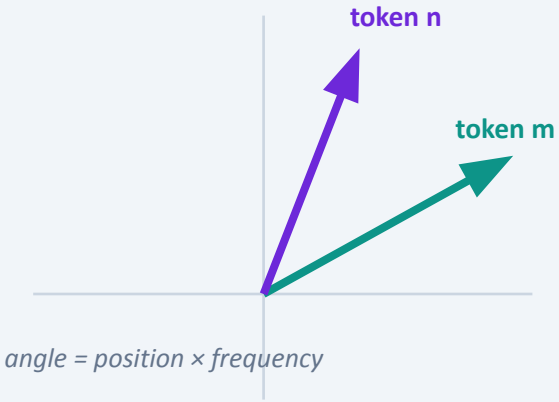
1 Positional encoding: RoPE

Attention ignores order, so we inject position — modern LLMs rotate queries & keys by an angle set by position.

Why: self-attention is permutation-invariant — “cat sat” and “sat cat” look identical unless we add position.

- Absolute: add a fixed or learned vector per position (original Transformer)
- Relative: bias attention by the distance (m − n)
- **RoPE (rotary): rotate Q and K — relative position falls out of the dot product**

Rotate by angle ∝ position



- Q·K ends up depending only on (m − n)
- No extra parameters to learn
- Different dimension-pairs rotate at different speeds

Why it matters here: RoPE is used in LLaMA, Qwen and most modern LLMs — and stretching its rotation frequencies (NTK-aware / YaRN) is how context windows get extended to 100k–1M tokens.

1 RoPE inside the attention formula

The rotation lands right in the QK^T score: rotate Q and K by position, and only the relative offset survives.

Standard self-attention: $\text{Attention}(Q, K, V) = \text{softmax}(\mathbf{Q K^T} / \sqrt{d}) V$

1 Rotate Q and K by position

$$q'_m = R_m q_m , \quad k'_n = R_n k_n$$

2 Take the score

$$q'^{T}_m k'_n = q^{T}_m R_{(n-m)} k_n \rightarrow \text{depends only on } (n - m)$$

3 Put it back into attention

$$\text{softmax}((R_m Q)(R_n K)^T / \sqrt{d}) V$$

The rotation R_m

$R(\theta) =$

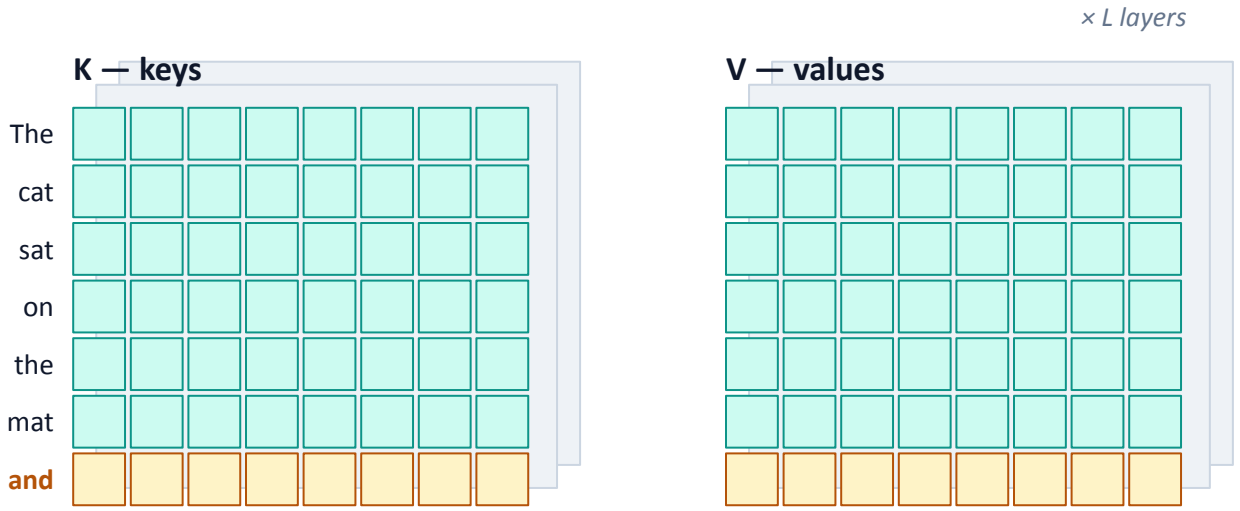
$$\begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

- applied to each (x_{2i} , x_{2i+1}) pair of the vector
- frequency $\theta_i = 10000^{(-2i/d)}$ — low dims spin fast, high dims slow
- $R_m^T R_n = R_{(n-m)}$ → relative position, no params

Position enters only as a rotation: the QK^T dot product turns it into a function of the distance $(n - m)$ — that is what makes RoPE relative and length-extrapolatable.

1 The KV cache — what compression really saves

During generation, past keys & values are cached so they aren't recomputed — but the cache keeps growing.



Each new token appends one row to K and one to V — earlier rows are cached, never recomputed.

rows = tokens (grow with context n) · columns = d dimensions · a separate K/V cache per layer (× L)

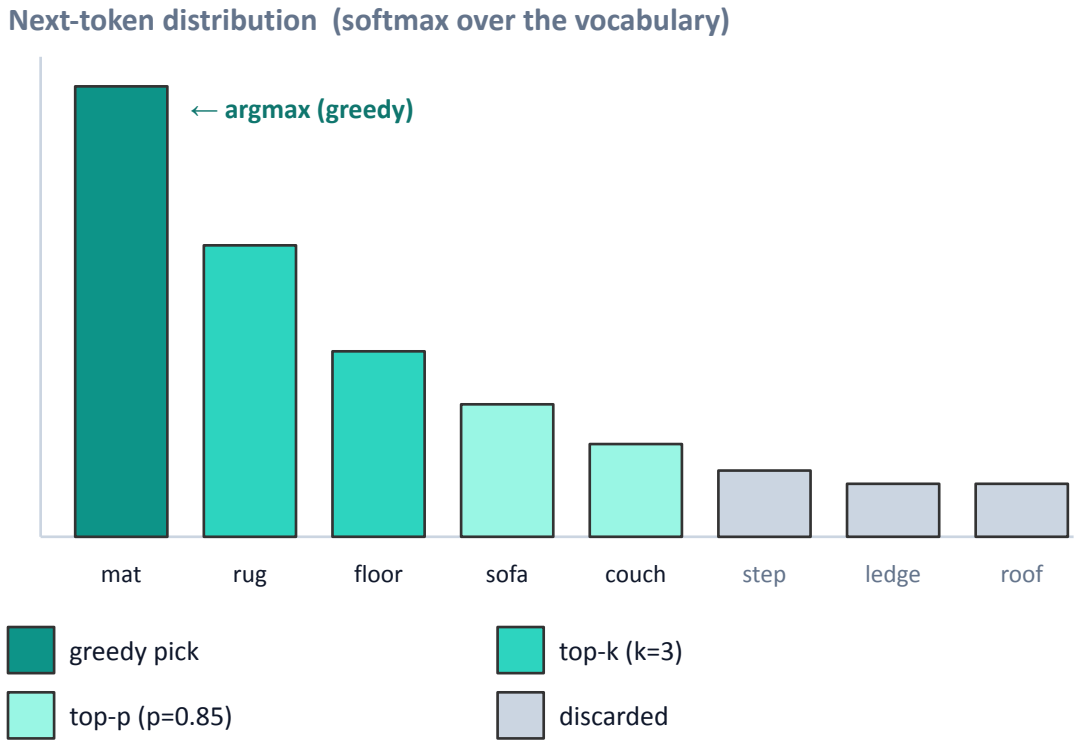
Cache size ≈

$$2 \cdot n \cdot d \cdot L$$

- n = context length
- d = model width
- L = number of layers
- **Shrinking n is the lever every inference-time compressor pulls**

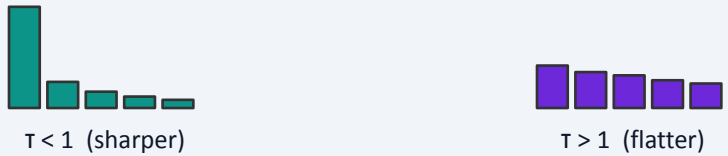
1 How LLMs decode — picking the next token

The model outputs a probability over the whole vocabulary; decoding is how we choose — trading accuracy for creativity.



Temperature τ reshapes the odds

τ → 0 = greedy · higher τ = more random



Deterministic

Greedy (argmax) · Beam search (keep B best). Reproducible; best for closed tasks like translation — but can be repetitive.

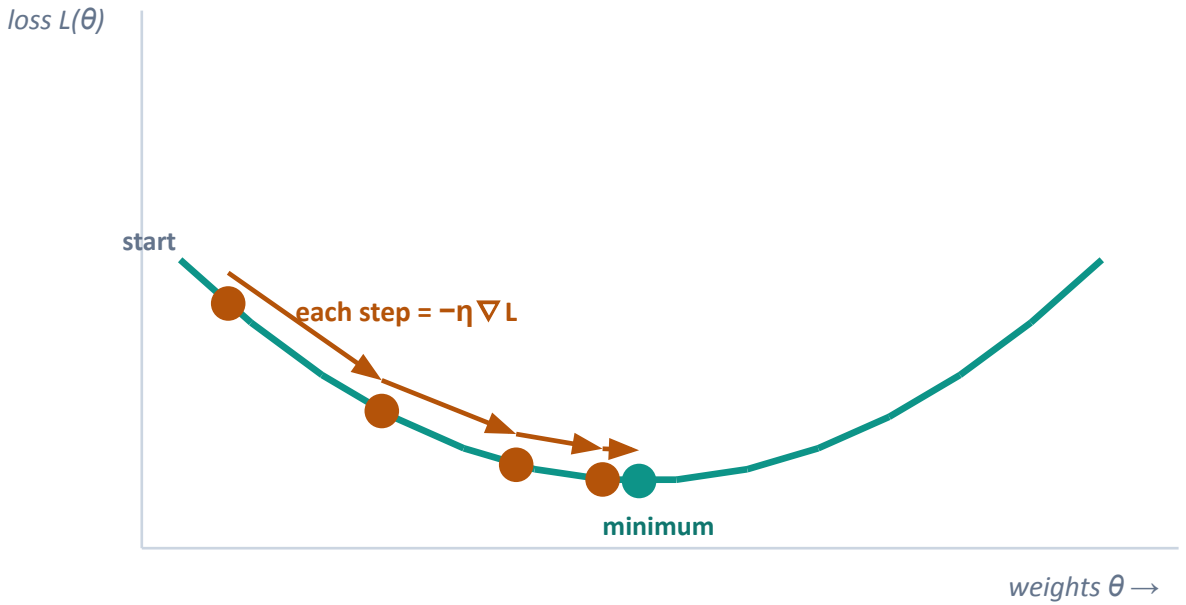
Sampling (stochastic)

Temperature · Top-k · Top-p / nucleus (+ repetition penalties). Diverse, natural text; the default for open-ended generation.

Autoregressive: pick a token, append it, repeat. Temperature & top-p set the creativity ↔ faithfulness dial — and constrained decoding (Part 4) masks this very distribution.

1 Optimization — learning by gradient descent

Training tunes the weights to shrink a loss, one downhill step at a time.



$$\theta \leftarrow \theta - \eta \cdot \nabla_\theta L(\theta)$$

Loss L(θ) how wrong the model is — cross-entropy on next-token prediction

Gradient ∇θL direction of steepest increase — so we step the opposite way

Learning rate η the step size: too big diverges, too small crawls

Backprop computes every gradient efficiently via the chain rule

SGD → Adam / AdamW: gradients from mini-batches; Adam adds momentum + per-parameter steps — the LLM default.

Same loop everywhere: pre-training runs it over the corpus; fine-tuning & LoRA reuse it on far fewer weights.

1 How LLMs are trained — and adapted

Learn general language once; specialize cheaply afterwards. Cheap specialization is what LoRA buys us.

1 · Pre-training

- **Objective: predict the next token**
- Loss: cross-entropy over a huge corpus (self-supervised — no labels)
- Optimizer: gradient descent + back-prop (Adam-style)
- Result: a general model that has read the internet

2 · Adaptation

- Supervised / instruction tuning on curated data
- Preference alignment (make it helpful & safe)
- **Full fine-tuning updates all weights → expensive**

LoRA

Parameter-efficient fine-tuning: freeze the model, learn a tiny low-rank adapter instead. Remember this — in Part 3 we compress a whole document into one such adapter.

PART 2

The long-context problem & RAG

The standard fix — and why it doesn't actually fix the cost.

- Why long context is hard
- RAG in one picture
- RAG's moving parts
- Why RAG isn't enough

2 Why long context is genuinely hard

Three independent failure modes push us toward retrieval — and then toward compression.

1

Cost

- Quadratic attention + a KV cache that grows with every token
- Long prompts are slow and memory-hungry to serve

2

Lost in the middle

- Models over-attend to the start & end of the context
- Evidence buried in the middle is often missed

3

Stale & private

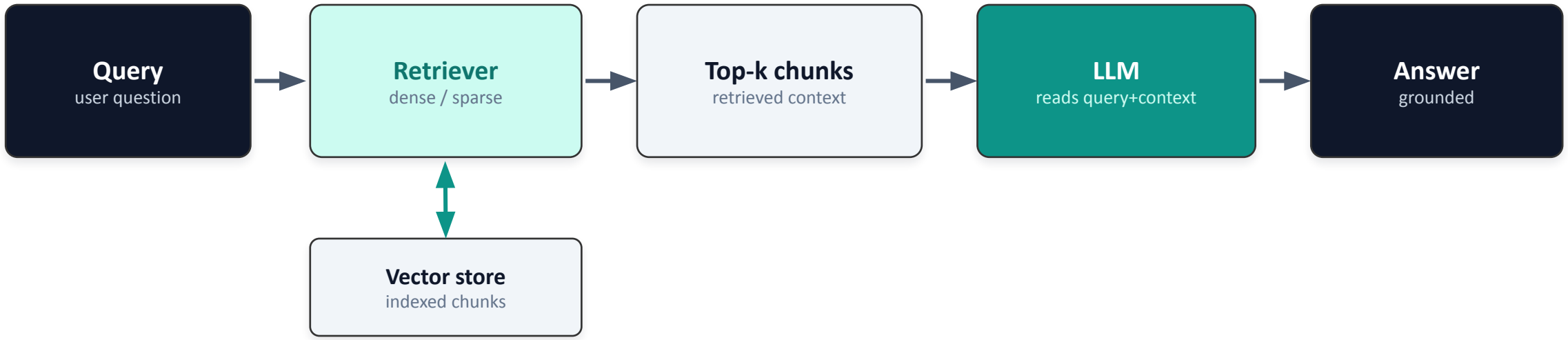
- Weights freeze at training time — no fresh facts
- Your private/company data was never in pre-training

Retrieval-augmented generation (RAG) attacks all three — by fetching the right text on demand.

"Lost in the middle": Liu et al., TACL 2024.

2 RAG in one picture

Retrieve relevant chunks from an external store, prepend them to the prompt, then generate.



Why it helps: fresh & private facts without retraining · answers can cite sources · smaller base model can punch above its weight.

2 RAG's moving parts

Each stage is an independent engineering choice — and each one can leak noise into the prompt.

1 Chunking
split documents into passages

2 Embedding model
text ⟶ dense vectors

3 Vector index
store & nearest-neighbour search

4 Retriever
dense · sparse · hybrid

5 Reranker
re-order candidates by relevance

6 Generator
LLM writes the final answer

2 Why RAG doesn't solve the problem

RAG relocates the problem into the prompt: you still pay for — and get distracted by — long context.

✕ **Cost is unchanged**
Retrieved chunks are long text ⟶ the quadratic + KV cost is right back.

✕ **Retrieved noise ⟶ hallucination**
Irrelevant passages distract the model and seed unfaithful answers.

✕ **The top-k dilemma & Temporality**
Small k misses evidence; large k bloats the prompt. No good setting.

✕ **Latency**
More context = slower first token, higher serving cost per query.

The fix: add a compression layer between retrieval and generation — keep the signal, drop the tokens.

PART 3

Compression techniques

The core of the talk: keep the meaning, spend far fewer tokens.

- Taxonomy: soft · hard · beyond
- Soft: Gist · xRAG · PISCO · OSCAR
- Hard: LLMLingua · Provence
- Into weights & architecture

3 A map of compression

Same goal — shrink the context — four different places to put the compressed information.

Soft

→ continuous vectors

Gist · xRAG
PISCO · OSCAR

Hard

→ shorter text

LLMLingua
Provence

Into weights

→ model parameters

Doc-to-LoRA

Into architecture

→ built-in memory

Recurrent Memory Transformer

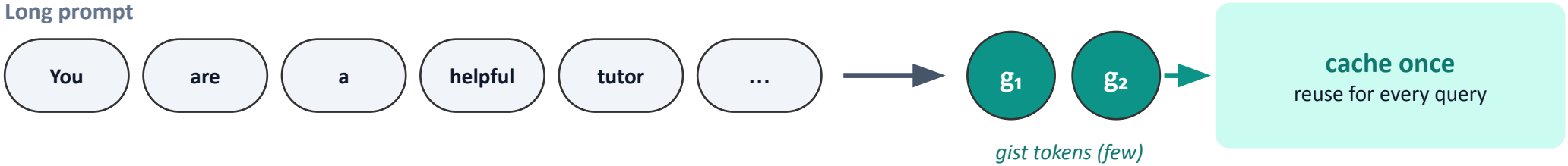
Every method trades along the same axes:

compression rate · accuracy loss · query-dependent vs query-agnostic · offline vs online · plug-and-play vs backbone-specific · needs training?

3 Gist Tokens

SOFT

A prompt can be squeezed into a few reusable vectors — the origin of every soft method.



How

- During instruction tuning, a modified attention mask forces the model to route the prompt's meaning through a few “gist” activation vectors
- Later tokens may attend to the gist vectors, but not to the original prompt
- Those vectors are cached and reused — the prompt is “read” once

up to 26×

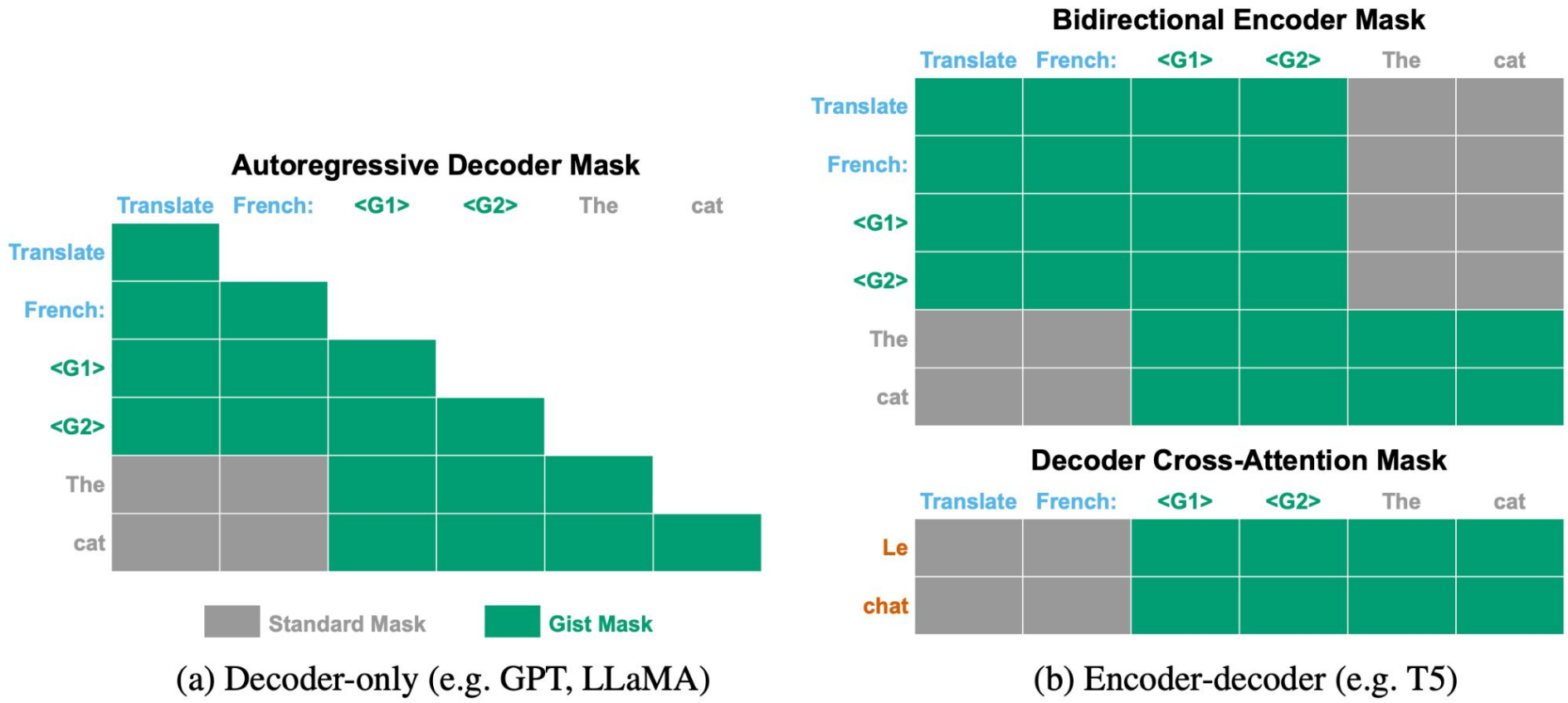
prompt compression

Establishes the whole premise: a prompt can live as a handful of vectors.

3 Gist Tokens

SOFT

A prompt can be squeezed into a few reusable vectors — the origin of every soft method.



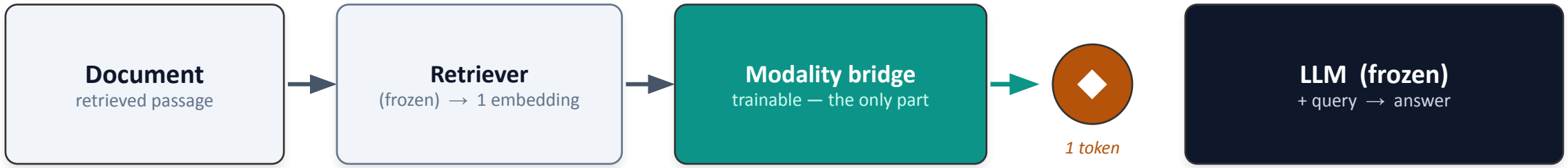
(a) Decoder-only (e.g. GPT, LLaMA)

(b) Encoder-decoder (e.g. T5)

3 xRAG — one token per document

SOFT

Represent an entire retrieved document as a single token the LLM can read.



Reuse what you already have

The retriever's document embedding — normally only used for search — becomes the input feature.

Only the bridge is trained

Retriever and LLM stay frozen → plug-and-play; reuse offline-built embeddings.

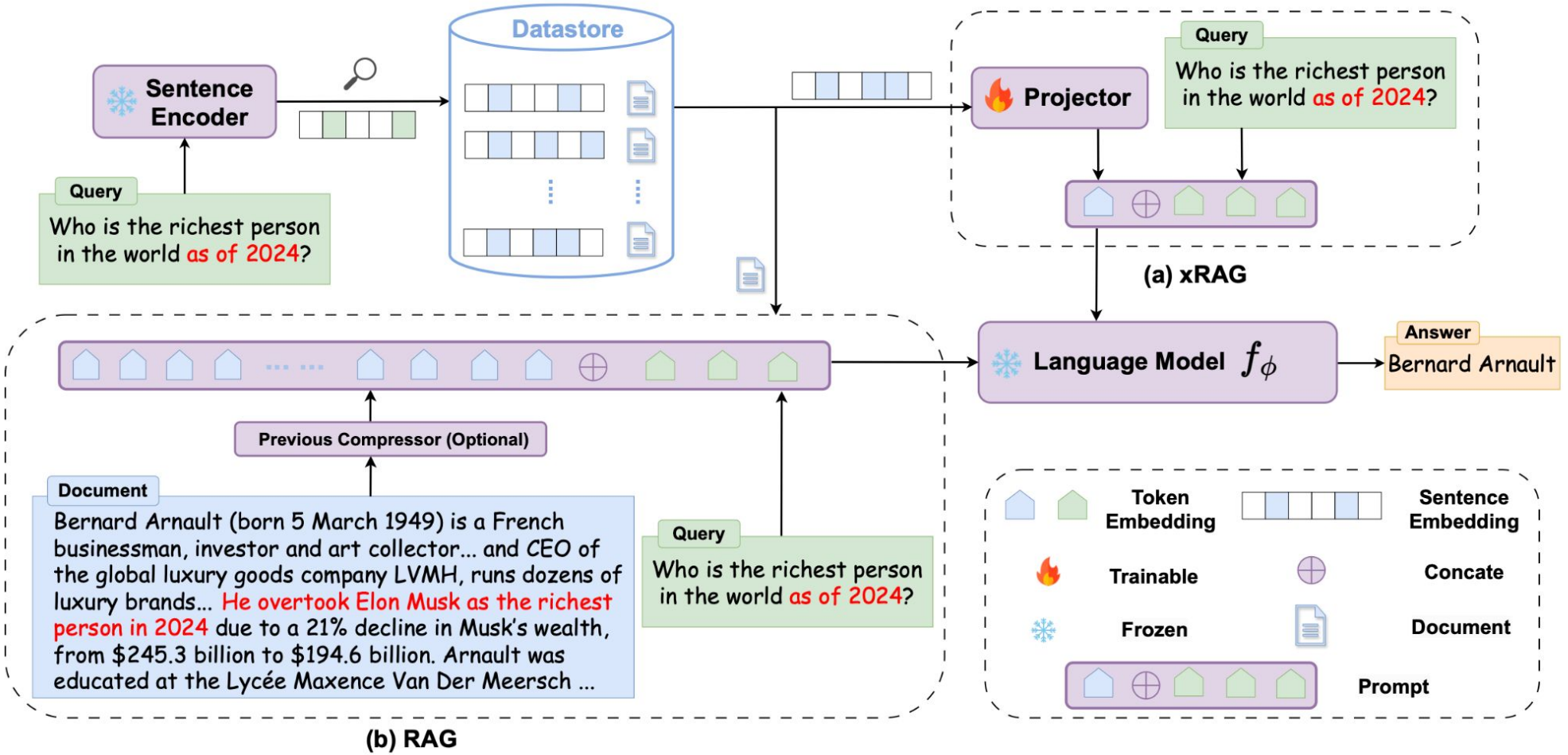
Extreme rate

A whole document = one token. ~3.5× fewer FLOPs, +10% over prior compressors.

3 xRAG — one token per document

SOFT

Represent an entire retrieved document as a single token the LLM can read.



3 PISCO — memory tokens + distillation

SOFT

Compress each document into 8 memory vectors — trained only by copying a teacher's answers.

trained by sequence-level distillation from a teacher LLM's answers

16×

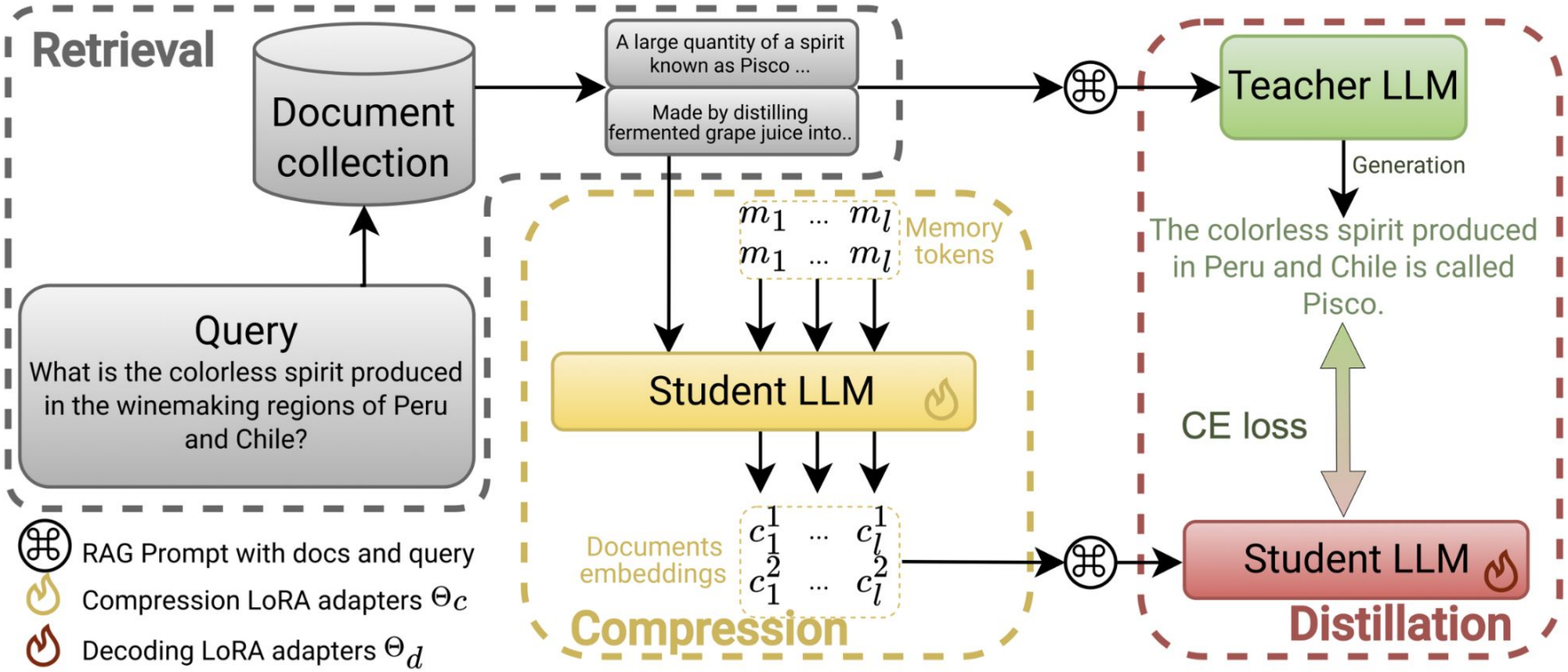
compression · 0–3% accuracy drop · ~5× faster

- Two LoRA adapters wrap one backbone: an encoder that compresses, a decoder that reads the memory tokens
- **Trained purely by distilling the teacher's own answers — no pre-training, no labels**
- Fine-tunes a 7–10B model in ~a day on a single GPU → practical for real RAG

3 PISCO — memory tokens + distillation

SOFT

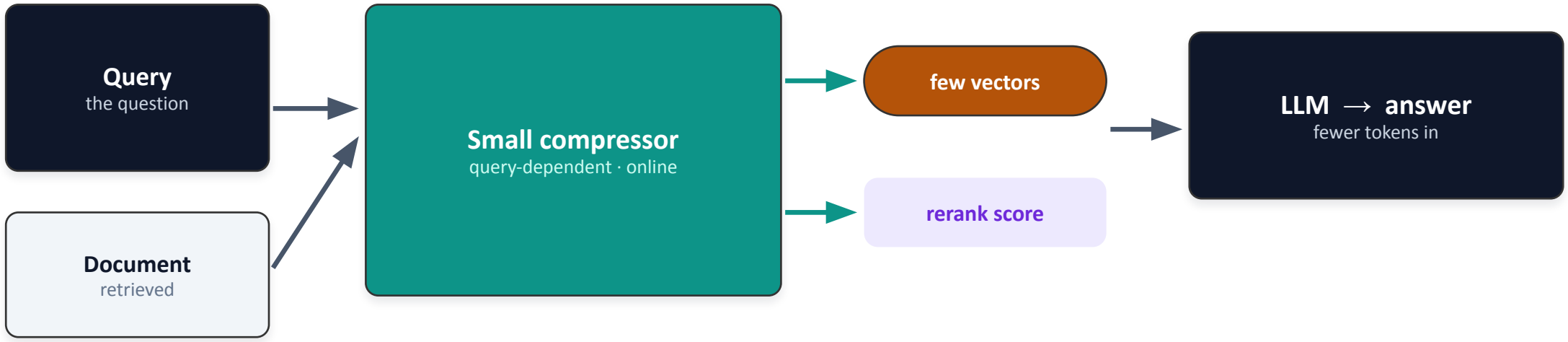
Compress each document into 8 memory vectors — trained only by copying a teacher's answers.



3 OSCAR — query-aware & online

SOFT

PISCO compresses documents once, ignoring the question. OSCAR compresses at query time, per question.



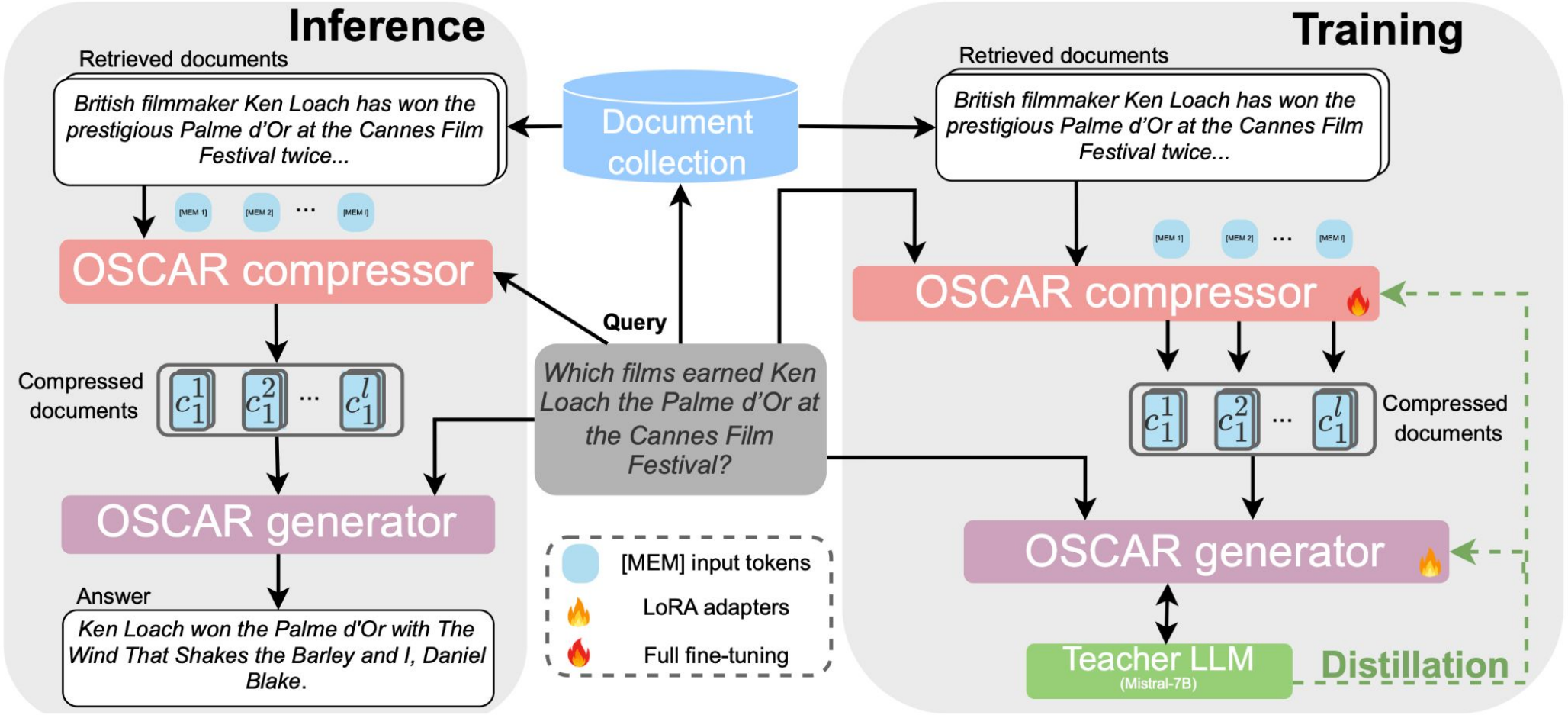
Reranking almost for free: pruning is folded into a step the pipeline already runs.

2–5× faster inference · backbones 1B–24B · little-to-no accuracy loss

3 OSCAR — query-aware & online

SOFT

PISCO compresses documents once, ignoring the question. OSCAR compresses at query time, per question.

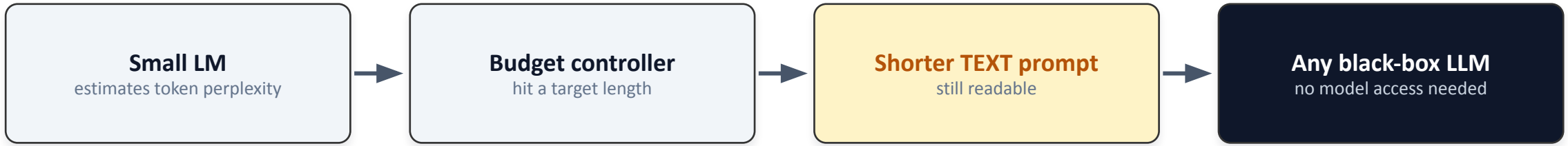


3 LLMLingua — drop the low-value tokens

HARD

Hard compression keeps natural language: a small model scores tokens and deletes the uninformative ones.

kept (high information) · dropped (low information)



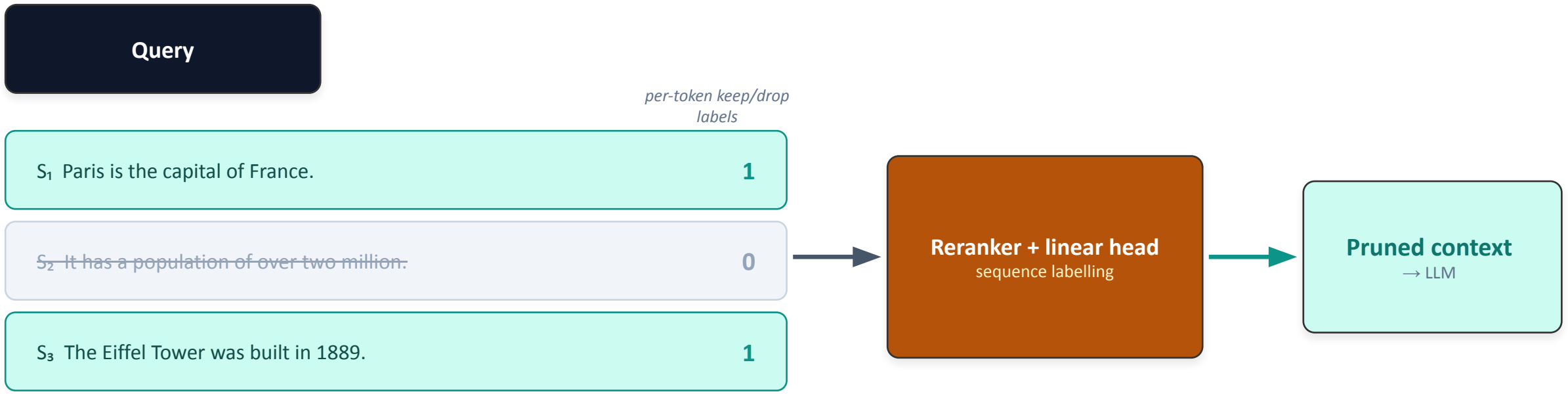
- Output is text → works with API-only models you can't modify
- Coarse-to-fine (drop sentences, then tokens) up to ~20×
- Follow-ups: LongLLMLingua (long-context ordering)
- LLMLingua-2: a fast distilled token classifier

LLMLingua · arXiv:2310.05736 (LongLLMLingua 2310.06839 · LLMLingua-2 2403.12968)

3 Provence — prune whole sentences

HARD

Frame pruning as per-token labelling on a reranker — and decide dynamically how much to cut.



Dynamic cuts 0 → all sentences as needed

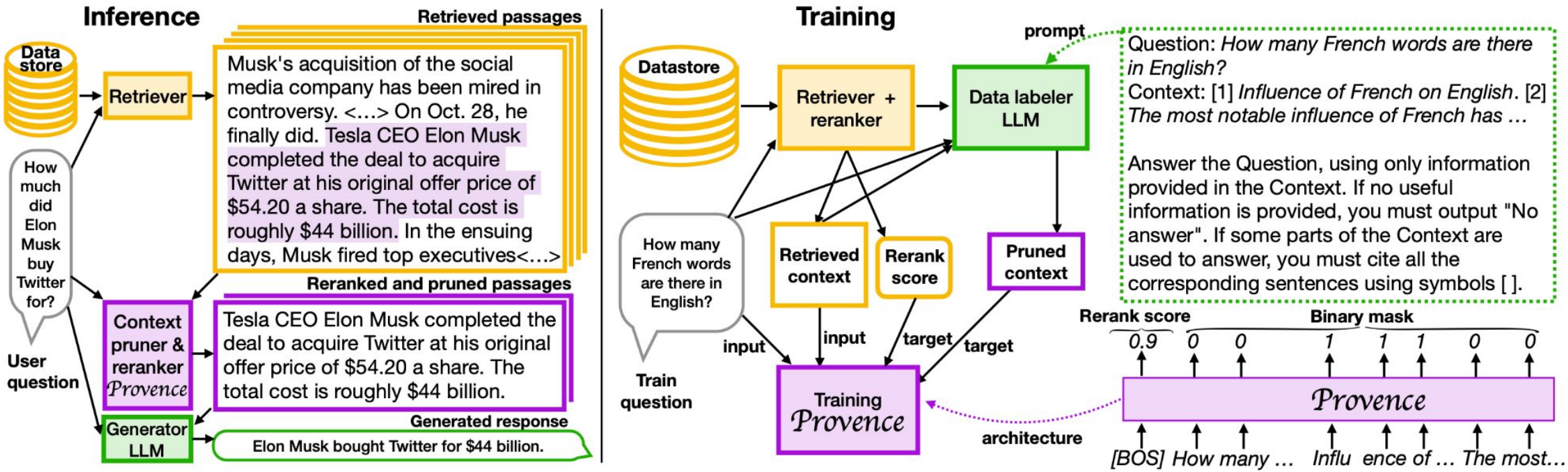
Unified pruning folded into reranking → ~zero-cost

~50% smaller with negligible accuracy loss

3 Provence — prune whole sentences

HARD

Frame pruning as per-token labelling on a reranker — and decide dynamically how much to cut.



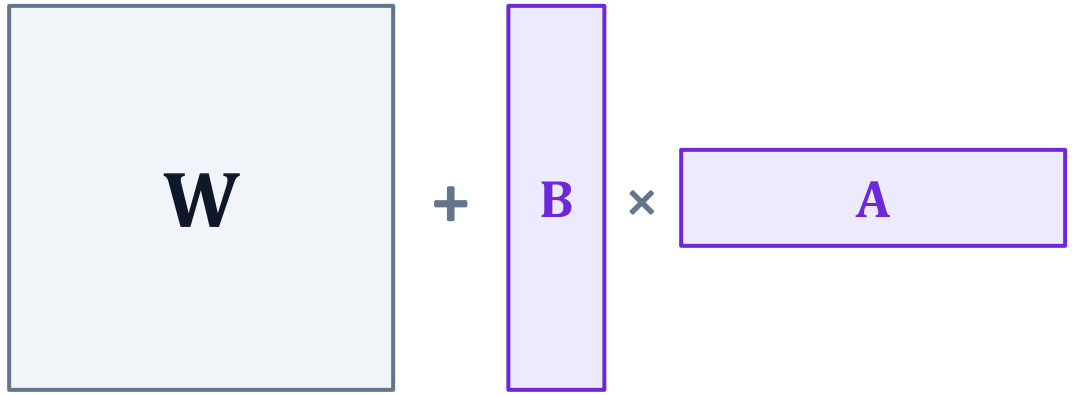
3 LoRA — a low-rank detour

MATH

Freeze the model; learn a tiny low-rank adapter instead — the object Doc-to-LoRA generates.

$$W + \Delta W \quad \text{with} \quad \Delta W = B \cdot A, \quad r \ll d$$

- Freeze the pretrained weight W (a d × d matrix)
- Learn only a low-rank update ΔW = B·A with rank r much smaller than d
- **Parameters: $d^2 \longrightarrow 2 \cdot d \cdot r$ (e.g. d = 4096, r = 8 ⟶ ~500× fewer)**
- The adapter is tiny and swappable — snap different behaviours onto one base model



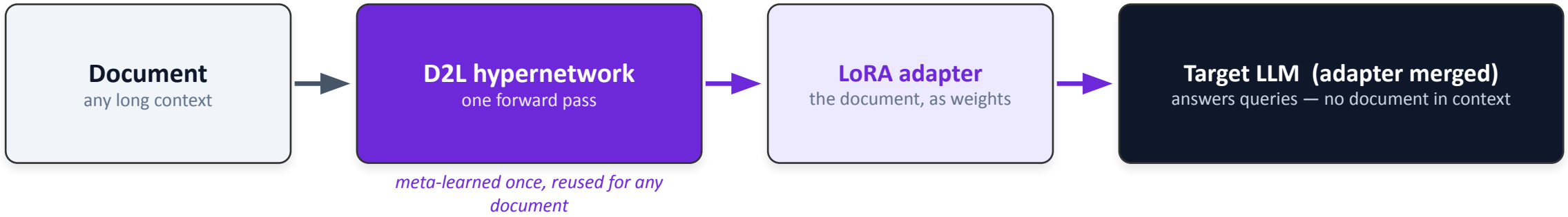
frozen · d × d

B·A = trainable, rank r

3 Doc-to-LoRA — context into weights

INTO WEIGHTS

The most radical option: turn a document into a LoRA adapter — then answer without the document at all.



Per-prompt distillation

- Train an adapter for each new document ⟶ slow, impractical
- Gradient descent every time you get a new context

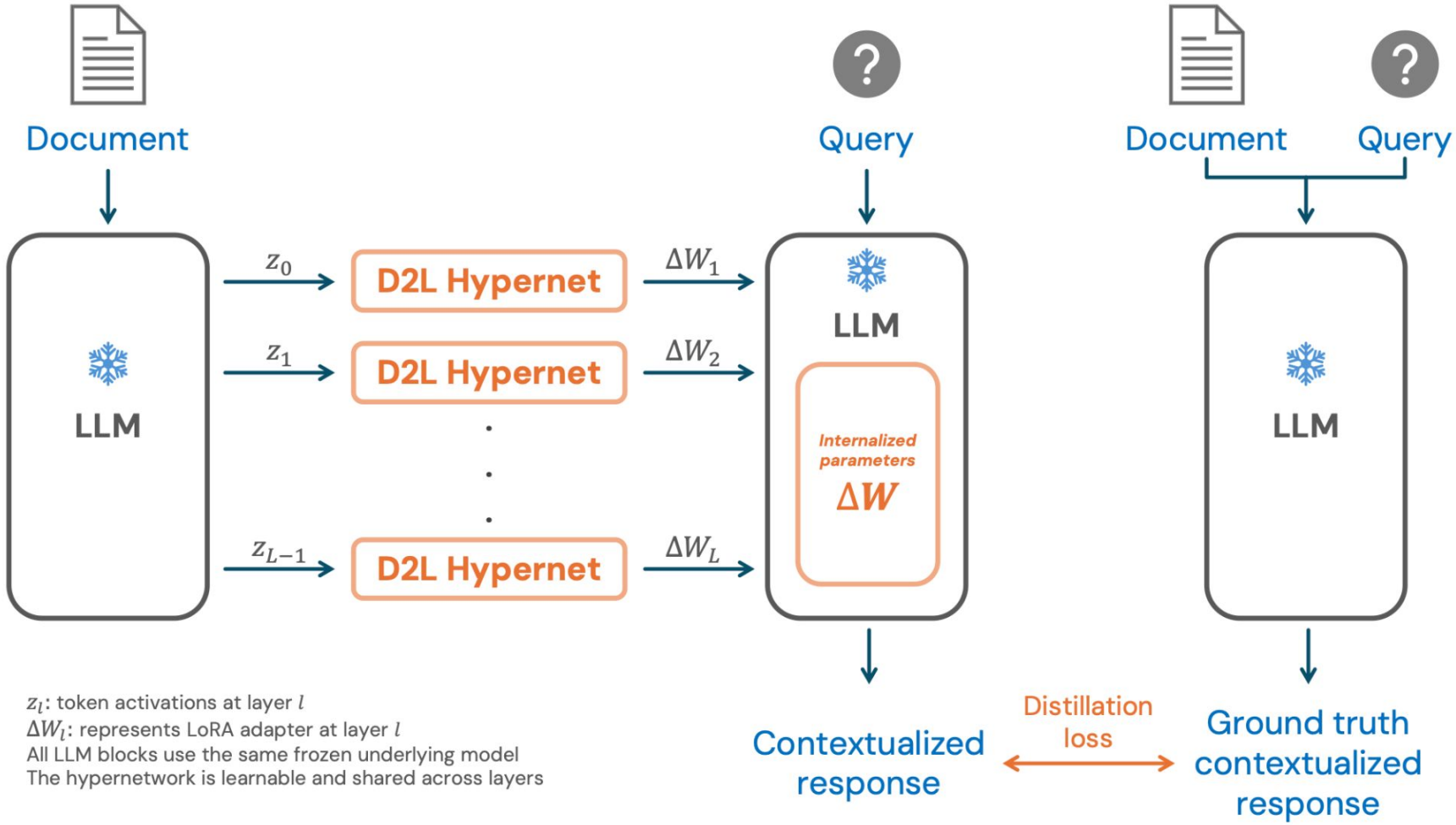
Doc-to-LoRA

- Amortized: one cheap forward pass "internalizes" the context — sub-second
- **Cuts KV-cache & latency; near-perfect needle-in-a-haystack at long lengths**

3 Doc-to-LoRA — context into weights

INTO WEIGHTS

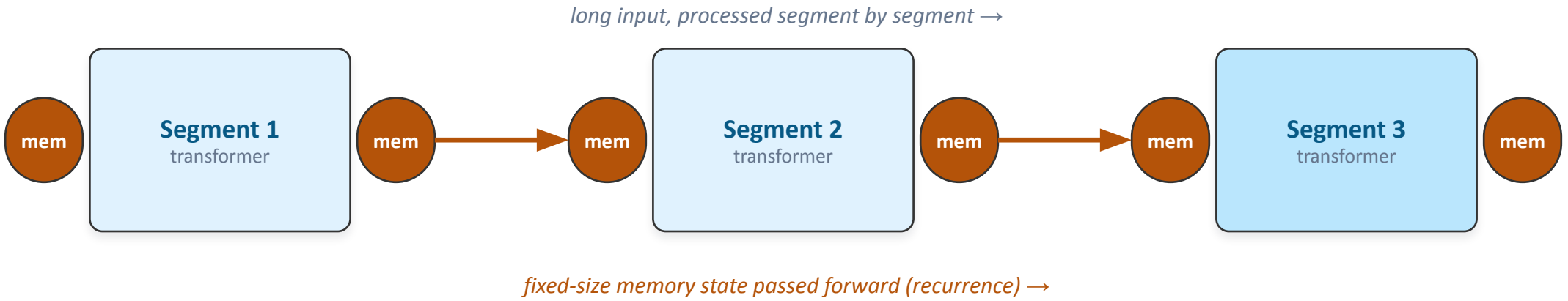
The most radical option: turn a document into a LoRA adapter — then answer without the document at all.



3 Recurrent Memory Transformer

ARCHITECTURE

Instead of compressing each document, redesign the model so a fixed-size memory carries the context.



Compression by construction: the whole past is squeezed into a small, fixed memory — not the $O(n^2)$ attention over everything.

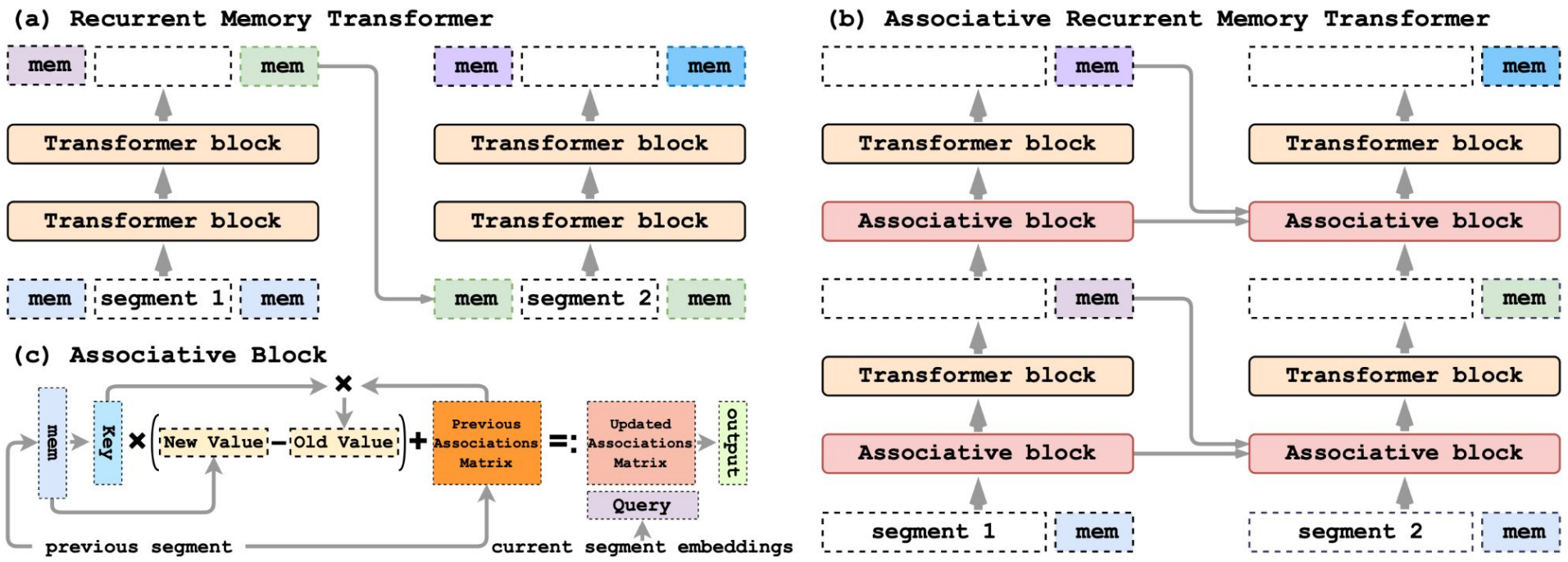
Effectively unbounded context: recurrence let scaled variants reach 1M+ tokens — a different bet than "bigger attention."

Recurrent Memory Transformer · arXiv:2207.06881 (scaling to 1M tokens · arXiv:2304.11062)

3 Recurrent Memory Transformer

ARCHITECTURE

Instead of compressing each document, redesign the model so a fixed-size memory carries the context.



3 Scoreboard — no free lunch

Every method buys token savings by paying somewhere: rate, training cost, query-awareness, or portability.

Method	Family	Context becomes	Query-dep?	Plug & play?	Headline effect
Gist Tokens	Soft	a few vectors	no	no	**~26× prompt**
xRAG	Soft	1 token / doc	no	yes	**~3.5× FLOPs**
PISCO	Soft	8 memory tokens	no	yes	**16× · 0–3% drop**
OSCAR	Soft (online)	query-aware vectors	yes	yes	**2–5× faster**
LLMLingua	Hard	shorter text	partly	yes (black-box)	**up to ~20×**
Provence	Hard	kept sentences	yes	yes	**~50% · ~0 loss**
Doc-to-LoRA	Into weights	a LoRA adapter	no	per backbone	**context ⟶ params**
RMT	Architecture	recurrent memory	n/a	no	**unbounded context**

PART 4

Reliability: the lossy tax

- Two kinds of hallucination
- Detecting them
- Structured output
- Constrained decoding = automata

Compression discards information — so what comes out needs checking.

4 Two kinds of hallucination

Compression stresses one of them in particular — faithfulness to the (now compressed) source.

Factuality

contradicts the world

- "The Eiffel Tower is in Rome."
- Wrong against general knowledge
- Mitigated mostly by retrieval / grounding

Faithfulness / grounding

contradicts the given source

- Source says 12% growth; the model answers "20%"
- The answer isn't supported by the context
- **Compress the source lossily → this risk goes up**

Why it belongs in this talk: if you crushed a document into 8 vectors or a LoRA, the model can answer confidently from a garbled version of it. (Also split intrinsic vs extrinsic — self-contradiction vs unsupported additions.)

4 Detecting hallucinations

Two broad families: ask another model, or read the model's own (un)certainty.

External checks

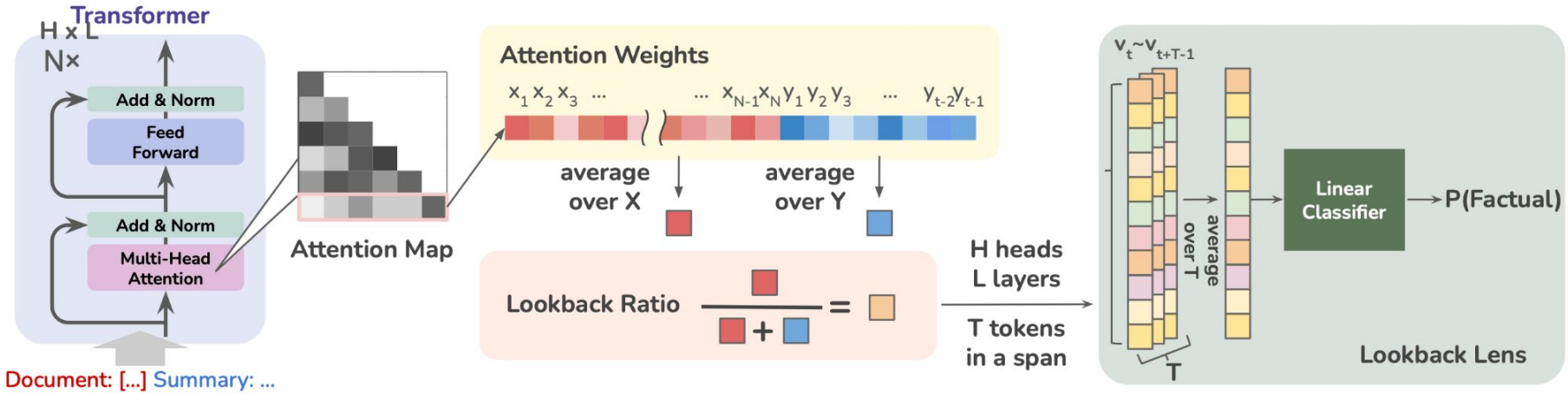
Encoder-based classification
Train an NLI / entailment model to flag claims the source doesn't support.

LLM-as-a-judge
Prompt a strong model to grade groundedness — flexible, but expensive and a bit circular.

Internal & statistical

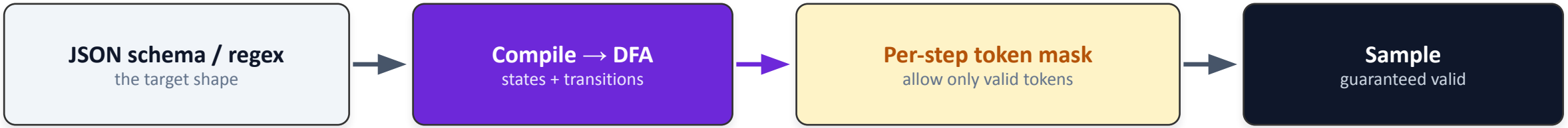
Uncertainty quantification
Token log-probs & predictive entropy; semantic entropy clusters meanings, not strings.

Self-consistency
Sample several answers; disagreement signals fabrication (SelfCheckGPT).



4 Structured output = walking an automaton

The maths payoff: constrained decoding intersects the model's output with a regular language.



current state → allowed next tokens

logits, masked to the DFA

invalid tokens get −∞ before sampling

In practice: Outlines (regex→FSM) · XGrammar · llguidance · API “structured output” modes · context-free grammars via pushdown automata · precomputed state→mask for near-free constraining.

4 Beyond regular: grammars & pushdown automata

Regex only covers regular languages. Real formats — JSON, code, expressions — are context-free or beyond, so the SoTA climbs the hierarchy.

PDA + check

Context-sensitive / semantic

CFG mask + external checker · e.g. type-correct code · declared-before-use · indentation

Monitor-Guided Decoding · PICARD · IterGen

PDA

Context-free

pushdown automaton (a stack) · e.g. nested JSON · XML · arithmetic · code syntax

XGrammar · llguidance · llama.cpp GBNF · DOMINO

DFA

Regular

finite automaton · e.g. regex · enums · fixed formats

Outlines (regex) · lm-format-enforcer ← previous slide

regular ⊂ context-free ⊂ context-sensitive (increasing expressive power)

Why a stack?

stack

A pushdown automaton = finite control + a stack. The stack tracks nesting depth (matched brackets, JSON) — which a DFA cannot do.

XGrammar — the CFG engine (MLSys 2025)

Compiles any context-free grammar to a pushdown automaton; precomputes context-independent token masks, resolves the rest at runtime → near-zero overhead. Ships in vLLM · SGLang · TensorRT-LLM.

Grammar-Aligned Decoding · ASAp (NeurIPS 2024)

Greedy masking guarantees validity but distorts the model's probabilities; ASAp provably re-aligns sampling to the LLM's true grammar-conditioned distribution.

XGrammar arXiv:2411.15100 · Grammar-Aligned Decoding (ASAp) NeurIPS 2024 · Monitor-Guided Decoding NeurIPS 2023

Four things to remember

1

Context is the bottleneck

Quadratic attention + a growing KV cache — not model size — is what limits long inputs.

2

Compression trades tokens for a little loss

Soft (vectors), hard (shorter text), hybrid — pick your point on the rate/accuracy/portability axes.

3

The frontier pushes past the prompt

Into the weights (Doc-to-LoRA) and into the architecture (RMT), not just fewer tokens in context.

4

Lossy → reliability is mandatory

Faithfulness detection and constrained (automaton-based) decoding are the companion, not an add-on.

Compress to cut cost · constrain & verify to control the loss you introduced.

References & further reading

- Mu, Li, Goodman. Learning to Compress Prompts with Gist Tokens. NeurIPS 2023. arXiv:2304.08467
- Jiang et al. LLMLingua. EMNLP 2023. arXiv:2310.05736 (LongLLMLingua 2310.06839 · LLMLingua-2 2403.12968)
- Cheng et al. xRAG: Extreme Context Compression for RAG with One Token. NeurIPS 2024. arXiv:2405.13792
- Louis, Déjean, Clinchant. PISCO: Pretty Simple Compression for RAG. ACL Findings 2025. arXiv:2501.16075
- Chirkova, Formal, Nikoulina, Clinchant. Provence: Context Pruning for RAG. ICLR 2025. arXiv:2501.16214
- Louis, Formal, Déjean, Clinchant. OSCAR: Online Soft Compression and Reranking. 2025. arXiv:2504.07109
- Hu et al. LoRA: Low-Rank Adaptation of LLMs. ICLR 2022. arXiv:2106.09685
- Charakorn et al. (Sakana AI). Doc-to-LoRA: Instantly Internalize Contexts. 2026. arXiv:2602.15902 (Text-to-LoRA: 2506.06105)
- Bulatov, Kuratov, Burtsev. Recurrent Memory Transformer. NeurIPS 2022. arXiv:2207.06881 (scaling: 2304.11062)
- Liu et al. Lost in the Middle. TACL 2024. arXiv:2307.03172
- Kuhn, Gal, Farquhar. Semantic Uncertainty. ICLR 2023. arXiv:2302.09664 (Farquhar et al., Nature 2024)
- Manakul et al. SelfCheckGPT. EMNLP 2023. arXiv:2303.08896
- Willard, Louf. Efficient Guided Generation for LLMs (Outlines). 2023. arXiv:2307.09702
- Dong et al. XGrammar: Efficient Structured Generation Engine. MLSys 2025. arXiv:2411.15100
- Park, Wang, Berg-Kirkpatrick. Grammar-Aligned Decoding (ASAp). NeurIPS 2024. arXiv:2405.21047

Diagrams are original schematics; figures were not reproduced from the source papers.

Skoltech'26

Skoltech Skolkovo Institute of Science and Technology

education research entrepreneurship industry community

Apply

Computational and Data Science and Engineering

3 years (full time)

The language of instruction is English

Tuition costs covered by Skoltech

Monthly scholarship is 75,000 rub (≈$680) for highest-scoring applicants

The application period for Skoltech PhD programs is now closed